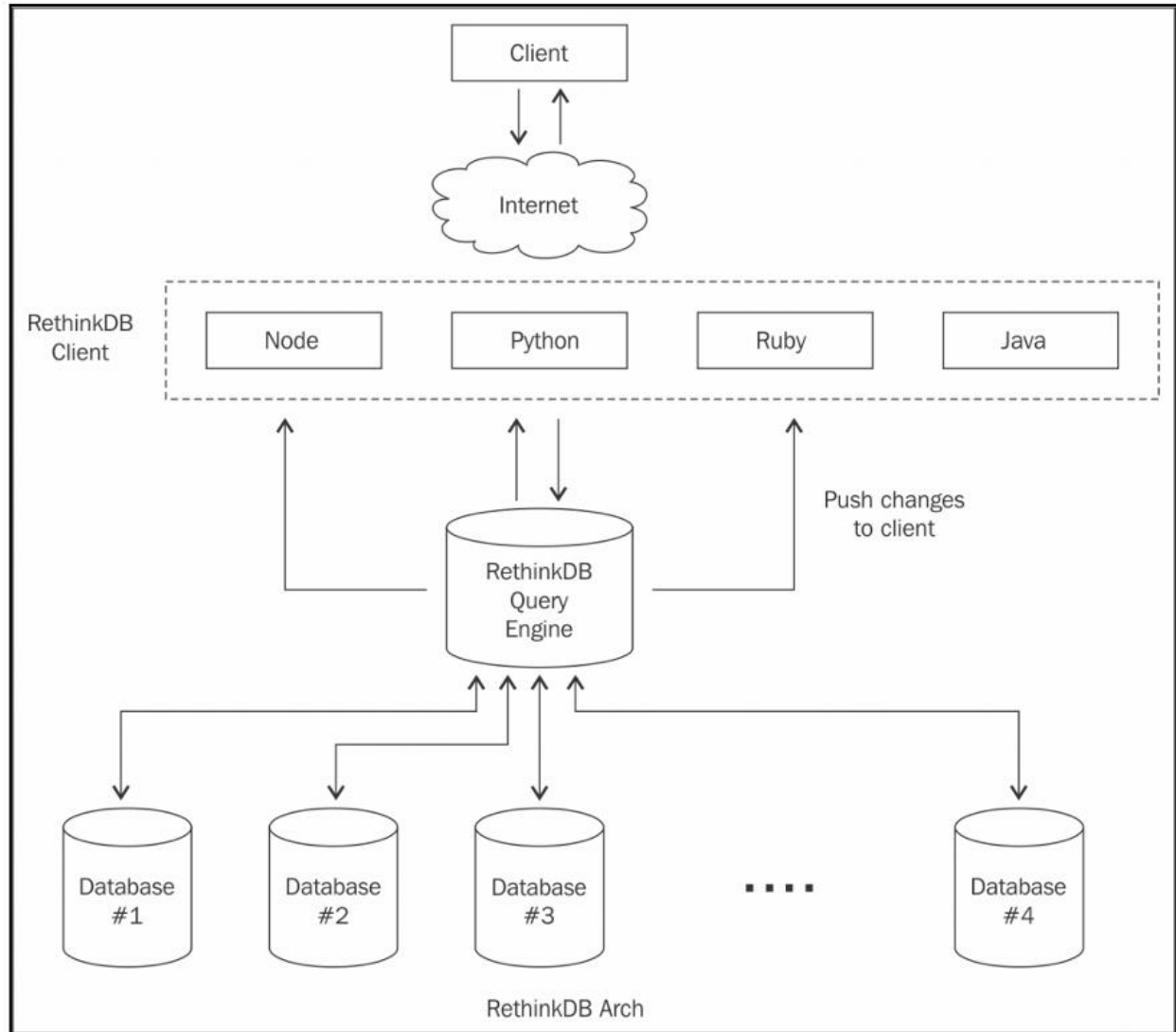




RethinkDB query engine

RethinkDB query handler, as name implies, performs query *execution and returns the response to the client*.

It does so by performing lot of *internal operations* such as *sorting, indexing, finding the cluster*, or *merging data from various clusters*. All of these operations are performed by RethinkDB query handler.

RethinkDB clusters

RethinkDB is a *distributed database* designed for high-performance, real-time operations.

RethinkDB manages distribution by clustering (*sharding or replication*).

RethinkDB clusters are just another instance of the main process of RethinkDB and store data.

Pushing changes to a RethinkDB client

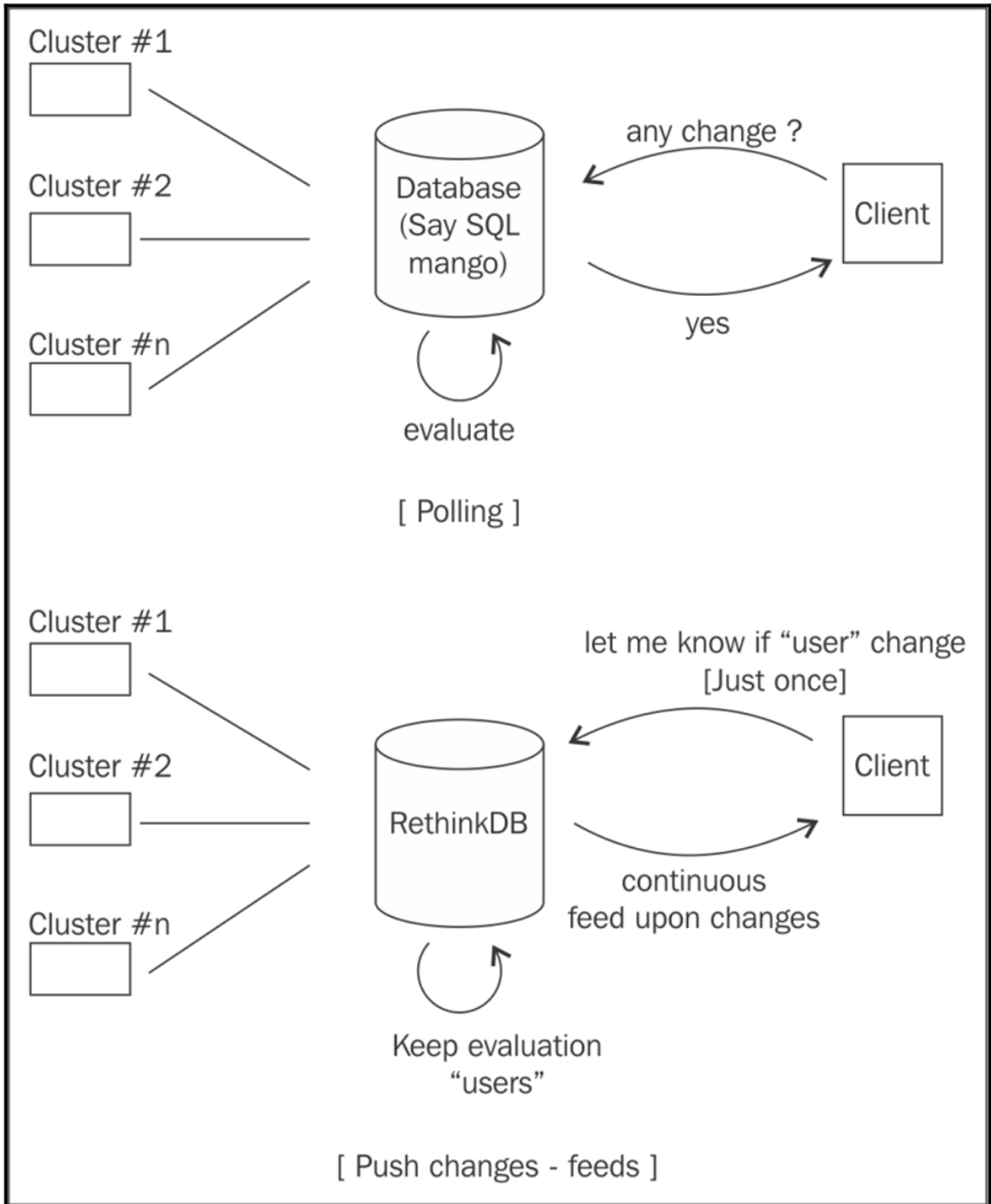
This is a revolutionary concept introduced by RethinkDB.

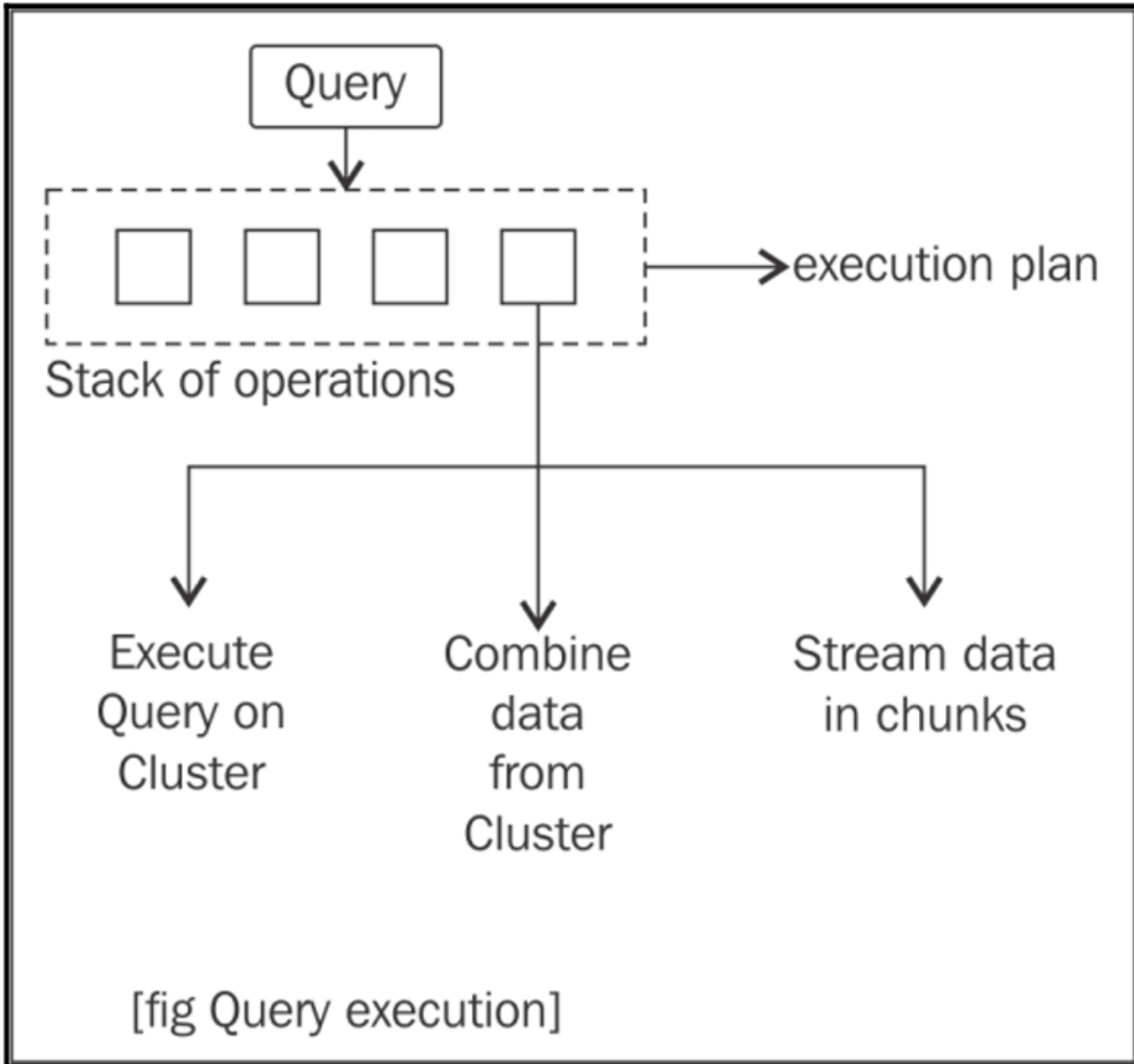
Consider this scenario: you are developing an application for the stock market where there are too many changes in a given amount of time.

Obviously, we are storing every entry in the database and making sure that other connected nodes or clients know about these changes.

Once you subscribe to a particular table to look for its changes, RethinkDB just keeps pushing the old and new values of changes to the connected client.

By “connected client,” I meant a *RethinkDB client* and not a web application client.





RethinkDB, upon arrival of a query, divides it into various stacks.

Each stack contains various methods and internal logic to perform its operation.

Each stack consists of various methods, but there are three core methods that play key roles:

- *The first method* decides how to execute the query or subset of the query on each server in a particular cluster

- *The second method* decides how to merge the data coming from various clusters in order to make sense of it
- *The third method*, which is very important, deals with transmission of that data in streams rather than as a whole

To speed up the process, these stacks are transported to every related server and each server begins to evaluate it in parallel to other servers.

This process runs recursively in order to merge the data to stream to the client.

The stack in the node grabs the data from the stack after it and performs its own method of execution and transformation.

The data from each server is then combined into a single result set and streamed to the client.

In order to speed up the process and maintain high performance, every query is completely parallelized across various relevant clusters.

Thus, every cluster then performs the query execution and the data is again merged together to make a single result set.

RethinkDB query engine maintains efficiency in the process too; for example, if a client only requests a certain result that is not in a shared or replicated server, it will not execute the parallel operation and just return the result set. This process is also referred to as lazy execution.

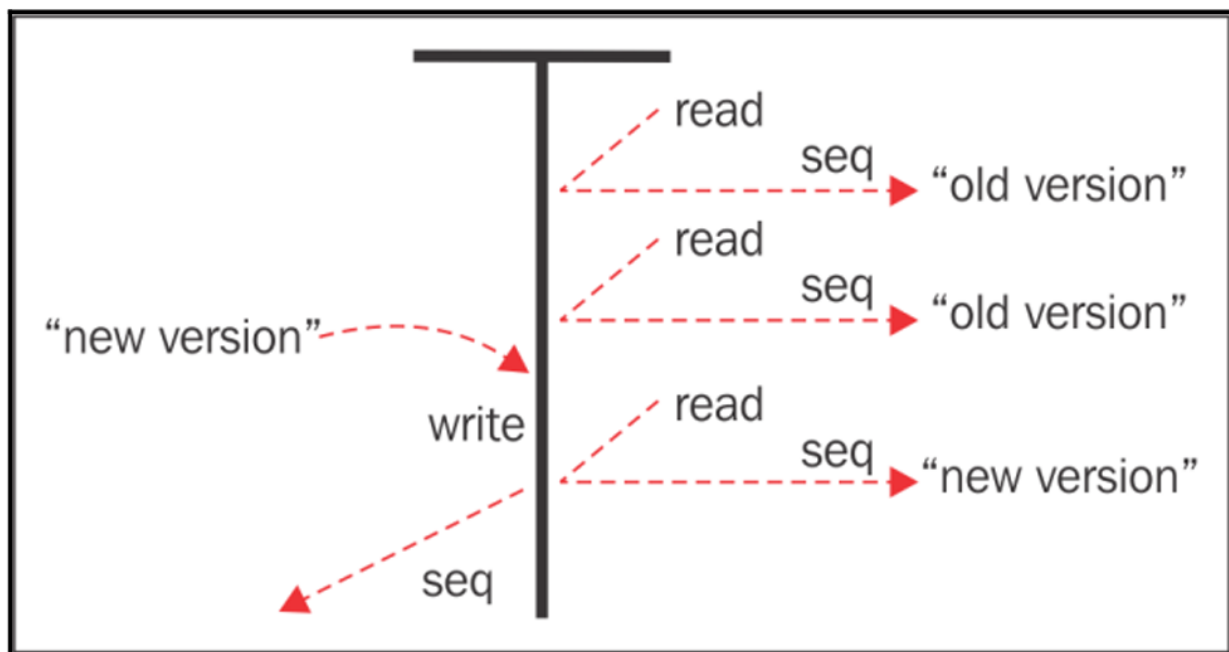
To maintain concurrency and high performance of query execution, RethinkDB uses block level *Multiversion Concurrency Control (MVCC)*.

If one user is reading some data while other users are writing on it, there is a high chance of inconsistent data, and to avoid that we use a **concurrency control algorithm**.

One of the simplest and commonly used methods method by SQL databases is to lock the transaction, that is, make the user wait if a write operation is being performed on the data.

This slows down the system, and since big data promises fast reading time, this simply won't work.

Multiversion concurrency control takes a different approach. Here each user will *see the snapshot of the data* (that is, *child copies of master data*), and if there are *some changes going on in the master copy*, then *the child copies or snapshot* will not get updated until the change has been committed:



RethinkDB does use block-level MVCC and this is how it works.

Whenever there is any update or write operation being performed during the read operation, RethinkDB takes a snapshot of each shard and maintains a different version of a block to make sure every read and write operation works in parallel.

RethinkDB does use exclusive locks on block level in case of multiple updates happening on the same document.

These locks are very short in duration because they all are cached; hence it always seems to be lock-free.

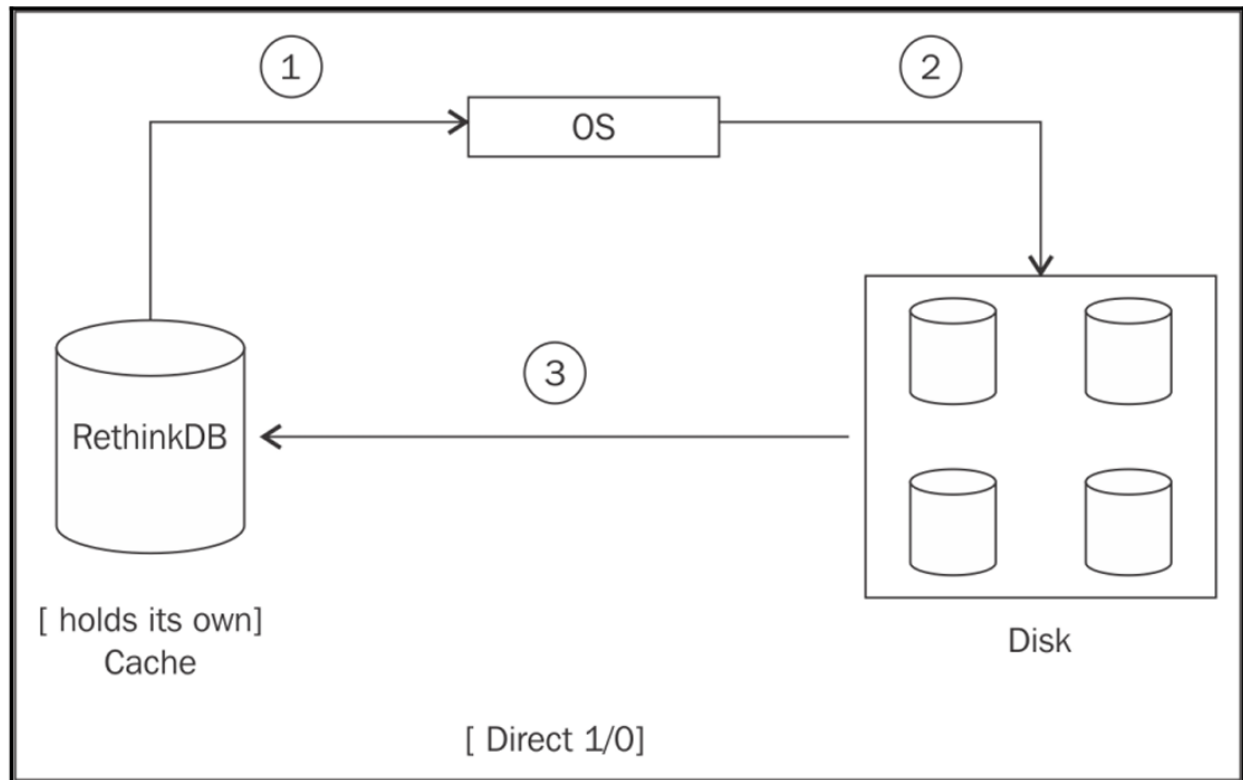
RethinkDB provides *atomicity of data as per the JSON document*.

This is different from other NoSQL systems; most NoSQL systems provide atomicity to each small operation done on the document before the actual commit.

RethinkDB does the opposite, it provides atomicity to a document *no matter what combination of operations is being performed*.

Direct I/O is a filesystem for those applications that want to avoid the buffering at the main memory and directly read files from disk.

When direct I/O is used, data is transferred directly to the application buffer instead of the memory buffer.



Data storage

RethinkDB uses a custom-built storage engine inspired by the Binary tree file system by Oracle (BTRFS). We have found the following promises by it:

- Fully concurrent garbage compactor
- Low CPU overhead
- Efficient multi-core operation
- SSD optimization
- Power failure recovery
- Data consistency in case of failure
- MVCC supports

Sharding and replication

Sharding is partitioning where the database is split across multiple smaller databases to improve [performance](#) and [reading time](#). In replication, we basically copy the database across multiple databases to provide a quicker look and less response time. Content delivery networks are the best examples of this. RethinkDB, just like other NoSQL databases, also uses sharding and replication to provide fast response and greater availability.

Sharding in RethinkDB

RethinkDB makes use of a range sharding algorithm to provide the sharding feature. It performs sharding on the table's primary key to partition the data. RethinkDB uses the table's primary key to perform all sharding operations and it cannot use any other keys to do so.

In RethinkDB, the [shard key](#) and [primary key are the same](#). Upon a request to create a new shard for a particular table, RethinkDB examines the table and tries to find out the optimal breakpoint to **create an even number of shards**.

The shards will be distributed across clusters automatically.

You can specify the sharding and replication settings at the time of creation of the table or alter it later. You *cannot specify the split point manually*; that is RethinkDB's job to do internally. You cannot have less server than you shard.

You can always visit the RethinkDB administrative screen to increase the number of shards or replicas:

Sharding and replication for test.users

Applying changes may cause the table to become temporarily unavailable.

1

shards

max: 1

1

replicas per shard

max: 1

►

Where's my data going?

Cancel

Apply configuration
